Thinking 4.1 思考并回答下面的问题：

- 内核在保存现场的时候是如何避免破坏通用寄存器的？
- 系统陷入内核调用后可以直接从当时的 `$a0-$a3` 参数寄存器中得到用户调用 `msyscall` 留下的信息吗？
- 我们是怎么做到让 `sys` 开头的函数“认为”我们提供了和用户调用 `msyscall` 时同样的参数的？
- 内核处理系统调用的过程对 `Trapframe` 做了哪些更改？这种修改对应的用户态的变化是什么？

根据 stackframe.h 头文件中的 SAVE_ALL 宏，内核在保存现场时，首先将原栈指针（sp）保存到 k0 寄存器中，然后获得新的栈指针，将 k0（原 sp 寄存器）和 v0 寄存器的值存入栈中，并利用 v0 寄存器将其他寄存器的值存入栈中。整个过程中除了 v0、sp、k0、k1 以外，其他未保存的通用寄存器的值不会改变，因为 v0 和 sp 的值已经被保存，而 k0 和 k1 是为操作系统保留的寄存器，不需要保护。因此，这个过程不会破坏通用寄存器。

当系统通过特权指令 syscall 陷入内核调用后，依次执行 .text.exec_vec3 段的异常处理函数，跳转到 handle_sys 函数执行 SAVE_ALL 和 CLI 宏汇编的过程，都不会改变 $a0-$a3 寄存器的值，所以这些寄存器可以直接使用。

在调用每个以 sys 开头的函数之前，我们从当前的内核栈中为 6 个参数申请特定的空间，并按顺序将参数存入栈中指定的位置。这样，sys 开头的函数就可以从 a0-a3 寄存器和内核栈中获得与用户调用 msyscall 时相同的参数。

在 Trapframe 中将 cp0_epc 增加 4，并将系统调用后的返回值存入 reg[2] 寄存器，使得系统调用结束后，用户态可以正确获得系统调用的返回值，并从发生系统调用的下一条指令开始执行。

Thinking 4.2 思考 `envid2env` 函数：为什么 `envid2env` 中需要判断 `e->env_id != envid` 的情况？如果没有这步判断会发生什么情况？

通过 **mkenvid** 函数可以知道，对于数组 **envs** 中相同位置的环境控制块，多次调用 **mkenvid** 生成的 **envid** 可能会有所不同。也就是说，虽然可以通过 **envid** 的低 11 位在数组 **envs** 中找到该 **id** 对应的进程控制块，但这并不能确保该环境控制块就是 **envid** 对应的控制块。需要进一步比较 **envid** 的高位信息是否相同。如果没有进行这种比较，可能会导致 **envid** 对应的环境控制块实际上并不存在，而函数仍然返回了相关的环境控制块信息，导致错误的结果。

Thinking 4.3 思考下面的问题，并对这个问题谈谈你的理解：请回顾 `kern/env.c` 文件中 `mkenvid()` 函数的实现，该函数不会返回 0，请结合系统调用和 IPC 部分的实现与 `envid2env()` 函数的行为进行解释。

由mkenvid()函数可知，最终返回值的第11位始终为一，所以该函数不会返回零。

Thinking 4.4 关于 fork 函数的两个返回值，下面说法正确的是：

A、fork 在父进程中被调用两次，产生两个返回值

B、fork 在两个进程中分别被调用一次，产生两个不同的返回值

C、fork 只在父进程中被调用了一次，在两个进程中各产生一个返回值

D、fork 只在子进程中被调用了一次，在两个进程中各产生一个返回值

C

Thinking 4.5 我们并不应该对所有的用户空间页都使用 duppage 进行映射。那么究竟哪些用户空间页应该映射，哪些不应该呢？请结合 kern/env.c 中 env_init 函数进行的页面映射、include/mmu.h 里的内存布局图以及本章的后续描述进行思考。

需要映射的是 0 到 USTACKTOP 范围内的空间。因为在此范围之上，USTACKTOP 到 UXSTACKTOP 之间的区域是用户进程的异常栈，而异常栈是用于处理异常的地方，不应被映射；UTOP 以上则是内核相关的页表，用户无权更改，也无需映射

Thinking 4.6 在遍历地址空间存取页表项时你需要使用到 vpd 和 vpt 这两个指针，请参考 user/include/lib.h 中的相关定义，思考并回答这几个问题：

- vpt 和 vpd 的作用是什么？怎样使用它们？
- 从实现的角度谈一下为什么进程能够通过这种方式来存取自身的页表？
- 它们是如何体现自映射设计的？
- 进程能够通过这种方式来修改自己的页表项吗？

vpd存放页目录基地址，基地址加页目录项偏移数即为va对应的页目录项；vpt为页表基地址，基地址加页表项偏移数即为va对应的页表项。

在entry.S中，定义了页表和页目录的虚拟地址，使每个进程的页表都能在UVPT中保存。

vpd指向(UVPT+(UVPT>>12)*4) ，这是自映射机制。

不能

Thinking 4.7 在 do_tlb_mod 函数中，你可能注意到了一个向异常处理栈复制 Trapframe 运行现场的过程，请思考并回答这几个问题：

- 这里实现了一个支持类似于“异常重入”的机制，而在什么时候会出现这种“异常重入”？
- 内核为什么需要将异常的现场 Trapframe 复制到用户空间？

在处理中断的过程中，再次发生中断。

我们是在用户进程中处理这个缺页中断，因此用户进程需要读取 Trapframe 的值来处理中断，并在中断结束后恢复现场，所以需要将其复制到用户空间。

Thinking 4.8 在用户态处理页写入异常，相比于在内核态处理有什么优势？ ■

体现了“微内核”的思想，旨在减少内核的工作量，并加快内核的运行速度。

Thinking 4.9 请思考并回答以下几个问题：

- 为什么需要将 `syscall_set_tlb_mod_entry` 的调用放置在 `syscall_exofork` 之前？
- 如果放置在写时复制保护机制完成之后会有怎样的效果？

■

因为在 syscall_exofork 过程中也可能发生缺页中断。

导致写时复制保护机制无法正常运行，无法处理缺页错误。